

Laboratorio 4. Juego con obstaculos en ESP32 + OLED

Isabella Montoya Diaz, Juan David Tamayo Garcia, Paula Torres Uribe

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	3
5. Estructura del código	3
6. Explicación de funciones principales	4
7. Comunicación	4
8. Interfaz de usuario	4
9. Procedimiento de prueba	5
10. Manejo de errores y seguridad	5

1. Identificación del proyecto

- **Nombre del proyecto:** Juegos en ESP32 + OLED
- **Integrantes del equipo:** Isabella Montoya Diaz, Juan David Tamayo Garcia, Paula Torres Uribe

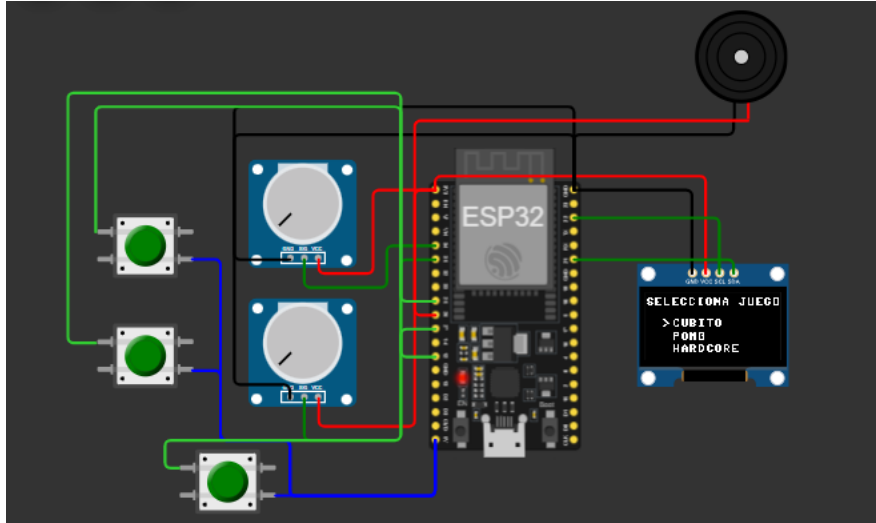
2. Resumen

Este proyecto implementa una serie de 3 juegos en ESP32 programados en MicroPython que se visualizan en una OLED SSD1306.

3. Descripción del hardware

- **ESP32** (modelo con ADC y pines GPIO)
- **OLED SSD1306 SCL** conectado al pin 22
- **OLED SSD1306 SDA** conectado al pin 21
- **Pulsador up** conectado al pin 12
- **Pulsador down** conectado al pin 25
- **Pulsador Menu/Start** conectado al pin 27
- **Buzzer** conectado al Pin 26
- **Potenciómetro jugador 1** conectado al Pin 34
- **Potenciómetro jugador 2** conectado al Pin 35
- **Fuente de alimentación:** 5V (USB)

Diagrama de conexión:



4. Descripción del software

- **Lenguaje usado:** MicroPython
- **Librerías:** machine, time, ssd1306, frambuf, unrandom
- **IDE:** Thonny

Flujo general del programa:

1. Inicialización del programa
2. Lectura de botones para seleccion del modo
3. Seleccion de modo mediante el menu
4. Lectura de los botones o potenciómetros segun el juego
5. Fin del juego
6. Regreso al menu principal

5. Estructura del código

- Archivo principal: `main.py`
- Se uso un modulo adicional el cual fue `ssd1036.py` .
- Código lineal dentro de un ciclo infinito que lee los botones en tiempo real y ejecuta la lógica del juego según el estado y modo activos.

6. Explicación de funciones principales

- `read_bottons()`: Antirebote
- `AABB(x1,y1,w1,h1,x2,y2,w2,h2)`: Detectar colisiones entre dos objetos.
- `draw_centered_text(o,y,txt)`: Dibuja texto centrado horizontalmente.
- `beep(freq=880,dur_ms,duty=512)`: Inicia un beep no bloqueante.
- `tone_update()`: Corta el beep cuando termina su duracion.
- `reset_pelota(direccion)`: Reinicia la pelota al centro con direccion al que recibe el punto.
- `mover_pelota()`: Movimiento de pelota y suma de puntos.
- `check_collision_paletas(px, py, pw, ph)`:Revisa si la pelota choco con una paleta.
- `draw_cubito()`: Si perdio muestra cuantos puntos obtuvo, si no dibuja el cubito y los obstaculos en la pantalla oled.
- `update(timer)`: Lee los botones, muestra el menu si perdio, muestra el menu de pausa, le genera el salto al cubo, mueve los obstaculos y revisa si colisiono con ellos.
- `draw_cubito_H()`: Si perdio en cubito hardcore muestra cuantos puntos obtuvo, si no dibuja el cubito y los obstaculos en la pantalla oled.
- `update_H(timer_H)`:Lee los botones, muestra el menu si perdio en cubito hardcore , muestra el menu de pausa, le genera el salto al cubo, genera que el cubo pueda agacharse, mueve los obstaculos, genera obstaculos de forma aleatoria y revisa si colisiono con ellos.
- `draw_menu`: Dibuja los tres modos de juego a seleccionar segun cual se selecciona, reinicia las variables y lo ejecuta.

7. Comunicación

Este proyecto utilizo comunicación serial, mediante i2C.

8. Interfaz de usuario

El usuario interactua con el sistema mediante tres botones. El botón menú/start permite elegir el modo de juego o pausarlo y reanudarlo durante la partida. El botón Up sirve para saltar y desplazarse en el menú hacia arriba, el botón Down hace que el Sprite se agache y también navegar hacia abajo. Para el modo de juego PONG, se tienen dos potenciómetros para que dos personas controlen las paletas al mismo tiempo. Por último se tiene la visualización en una OLED del menú y la ejecución de los juegos.

9. Procedimiento de prueba

1. Conectar el ESP32 al computador.
2. Cargar el código en el ESP32.
3. Cargar y abrir la libreria `ssd1306.py`.
4. Visualizar en la pantalla oled el menu y seleccionar juego.
5. Iniciar juego.
6. Probar botones y potenciómetros.
7. Pausar y cambiar de juego.

10. Manejo de errores y seguridad

- El voltaje del potenciómetro no debe superar el nivel tolerado por el ADC.
- Configurar los tiempos de refresco de pantalla para que el juego sea fluido.